Artificial Intelligence

Detect hallucinations for RAG-based systems

by Zainab Afolabi, Aiham Taleb, Nikita Kozodoi, and Liza (Elizaveta) Zinovyeva | on 16 MAY 2025 | in Amazon Bedrock, Amazon Bedrock Guardrails, Best Practices, Generative AI | Permalink | Comments | Share

With the rise of generative AI and knowledge extraction in AI systems, Retrieval Augmented Generation (RAG) has become a prominent tool for enhancing the accuracy and reliability of AI-generated responses. RAG is as a way to incorporate additional data that the large language model (LLM) was not trained on. This can also help reduce generation of false or misleading information (hallucinations). However, even with RAG's capabilities, the challenge of AI hallucinations remains a significant concern.

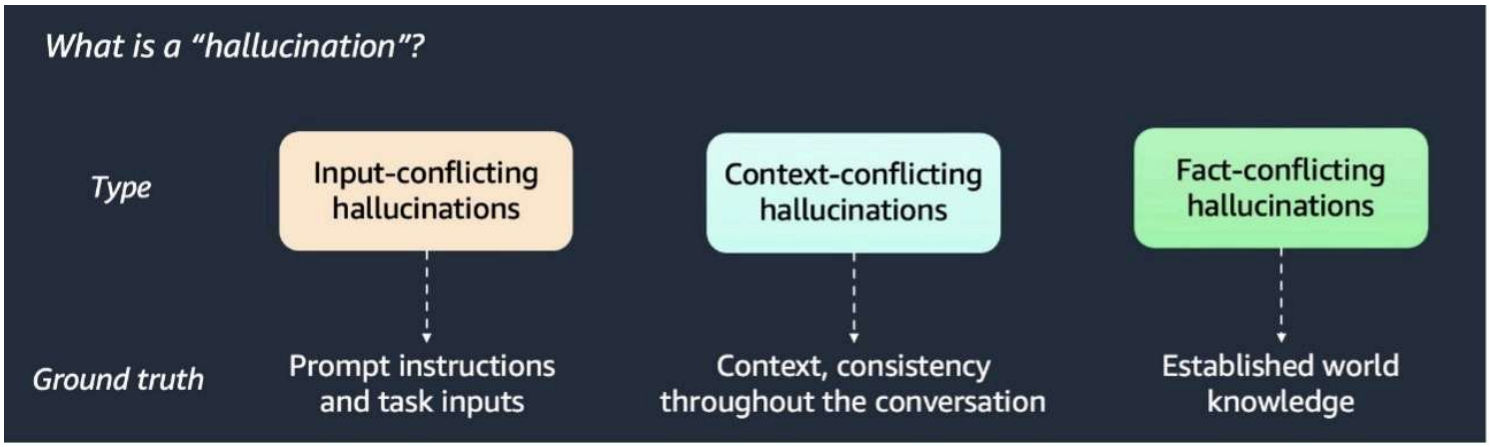
As AI systems become increasingly integrated into our daily lives and critical decision-making processes, the ability to detect and mitigate hallucinations is paramount. Most hallucination detection techniques focus on the prompt and the response alone. However, where additional context is available, such as in RAG-based applications, new techniques can be introduced to better mitigate the hallucination problem.

This post walks you through how to create a basic hallucination detection system for RAG-based applications. We also weigh the pros and cons of different methods in terms of accuracy, precision, recall, and cost.

Although there are currently many new state-of-the-art techniques, the approaches outlined in this post aim to provide simple, user-friendly techniques that you can quickly incorporate into your RAG pipeline to increase the quality of the outputs in your RAG system.

Solution overview

Hallucinations can be categorized into three types, as illustrated in the following graphic.



Scientific literature has come up with multiple hallucination detection techniques. In the following sections, we discuss and implement four prominent approaches to detecting hallucinations: using an LLM prompt-based detector, semantic similarity detector, BERT stochastic checker, and token similarity detector. Finally, we compare approaches in terms of their performance and latency.

Prerequisites

To use the methods presented in this post, you need an AWS account with access to Amazon SageMaker, Amazon Bedrock, and Amazon Simple Storage Service (Amazon S3).

From your RAG system, you will need to store three things:

- **Context –** The area of text that is relevant to a user's query
- **Question –** The user's query
- **Answer –** The answer provided by the LLM

The resulting table should look similar to the following example.

question	context	answer
What are cocktails?	Cocktails are alcoholic mixed…	Cocktails are alcoholic mixed…
What are cocktails?	Cocktails are alcoholic mixed…	They have distinct histories…
What is Fortnite?	Fortnite is a popular video…	Fortnite is an online multi…
What is Fortnite?	Fortnite is a popular video…	The average Fortnite player spends…

Approach 1: LLM-based hallucination detection

We can use an LLM to classify the responses from our RAG system into context-conflicting hallucinations and facts. The aim is to identify which responses are based on the context or whether they contain hallucinations.

This approach consists of the following steps:

1. Create a dataset with questions, context, and the response you want to classify.
2. Send a call to the LLM with the following information:
    1. Provide the statement (the answer from the LLM that we want to classify).
    2. Provide the context from which the LLM created the answer.
    3. Instruct the LLM to tag sentences in the statement that are directly based on the context.
3. Parse the outputs and obtain sentence-level numeric scores between 0–1.
4. Make sure to keep the LLM, memory, and parameters independent from the ones used for Q&A. (This is so the LLM can't access the previous chat history to draw conclusions.)
5. Tune the decision threshold for the hallucination scores for a specific dataset based on domain, for example.
6. Use the threshold to classify the statement as hallucination or fact.

Create a prompt template

To use the LLM to classify the answer to your question, you need to set up a prompt. We want the LLM to take in the context and the answer, and determine from the given context a hallucination score. The score will be encoded between 0 and 1, with 0 being an answer directly from the context and 1 being an answer with no basis from the context.

The following is a prompt with few-shot examples so the LLM knows what the expected format and content of the answer should be:

```
prompt = """\n\nHuman: You are an expert assistant helping human to check if statements are ba
 Your task is to read context and statement and indicate which sentences in the statement are

Provide response as a number, where the number represents a hallucination score, which is a fl
Set the float to 0 if you are confident that the sentence is directly based on the context.
Set the float to 1 if you are confident that the sentence is not based on the context.
If you are not confident, set the score to a float number between 0 and 1. Higher numbers repr

Do not include any other information except for the the score in the response. There is no nee

<example>
Context: Amazon Web Services, Inc. (AWS) is a subsidiary of Amazon that provides on-demand clo
Statement: 'AWS is Amazon subsidiary that provides cloud computing services.'
Assistant: 0.05
</example>

<example>
Context: Amazon Web Services, Inc. (AWS) is a subsidiary of Amazon that provides on-demand clo
Statement: 'AWS revenue in 2022 was $80 billion.'
Assistant: 1
</example>

<example>
Context: Monkey is a common name that may refer to most mammals of the infraorder Simiiformes,
Statement:'Average monkey is 2 meters high and weights 100 kilograms.'
Assistant: 0.9
</example>

Context: {context}
Statement: {statement}

\n\nAssistant: [
    """
    ### LANGCHAIN CONSTRUCTS
    # prompt template
    prompt_template = PromptTemplate(
        template=prompt,
        input_variables=["context", "statement"],
    )
```

Configure the LLM

To retrieve a response from the LLM, you need to configure the LLM using Amazon Bedrock, similar to the following code:

```
def configure_llm() -> Bedrock:

    model_params= { "answer_length": 100, # max number of tokens in the answer
        "temperature": 0.0, # temperature during inference
        "top_p": 1, # cumulative probability of sampled tokens
        "stop_words": [ "\n\nHuman:", "]", ], # words after which the generation is stopped
                    }
    bedrock_client = boto3.client(
            service_name="bedrock-runtime",
            region_name="us-east-1",
            )

    MODEL_ID = "anthropic.claude-3-5-sonnet-20240620-v1:0"

    llm = Bedrock(
        client=bedrock_client,
        model_id=MODEL_ID,
        model_kwargs=model_params,
        )

    return llm
```

Get hallucination classifications from the LLM

The next step is to use the prompt, dataset, and LLM to get hallucination scores for each response from your RAG system. Taking this a step further, you can use a threshold to determine whether the response is a hallucination or not. See the following code:

```
def get_response_from_claude(context: str, answer: str, prompt_template: PromptTemplate, llm:

    llm_chain = LLMChain(llm=llm, prompt=prompt_template, verbose=False)
    # compute scores
    response = llm_chain(
        {"context": context, "statement": str(answer)}
    )
    try:
        scores = float(scores)
    except Exception:
        print(f"Could not parse LLM response: {scores}")
        scores = 0
    return scores
```

Approach 2: Semantic similarity-based detection

Under the assumption that if a statement is a fact, then there will be high similarity with the context, you can use semantic similarity as a method to determine whether a statement is an input-conflicting hallucination.

This approach consists of the following steps:

1. Create embeddings for the answer and the context using an LLM. (In this example, we use the Amazon Titan Embeddings model.)
2. Use the embeddings to calculate similarity scores between each sentence in the answer and the (In this case, we use cosine similarity as a distance metric.) Out-of-context (hallucinated sentences) should have low similarity with the context.
3. Tune the decision threshold for a specific dataset (such as domain dependent) to classify hallucinating statements.

Create embeddings with LLMs and calculate similarity

You can use LLMs to create embeddings for the context and the initial response to the question. After you have the embeddings, you can calculate the cosine similarity of the two. The cosine similarity score will return a number between 0 and 1, with 1 being perfect similarity and 0 as no similarity. To translate this to a hallucination score, we need to take 1—the cosine similarity. See the following code:

```
def similarity_detector(
    context: str,
    answer: str,
    llm: BedrockEmbeddings,
) -> float:
    """
    Check hallucinations using semantic similarity methods based on embeddings<br /><br />

    Parameters
    ----------
    context : str
        Context provided for RAG
    answer : str
        Answer from an LLM
    llm : BedrockEmbeddings
        Embeddings model

    Returns
    -------
    float
        Semantic similarity score
    """

    if len(context) == 0 or len(answer) == 0:
        return 0.0
    # calculate embeddings
    context_emb = llm.embed_query(context)
    answer_emb = llm.embed_query(answer)
    context_emb = np.array(context_emb).reshape(1, -1)
    answer_emb = np.array(answer_emb).reshape(1, -1)
    sim_score = cosine_similarity(context_emb, answer_emb)
    return 1 - sim_score[0][0]
```

Approach 3: BERT stochastic checker

The BERT score uses the pre-trained contextual embeddings from a pre-trained language model such as BERT and matches words in candidate and reference sentences by cosine similarity. One of the traditional metrics for evaluation in natural language processing (NLP) is the BLEU score. The BLEU score primarily measures precision by calculating how many n-grams (consecutive tokens) from the candidate sentence appear in the reference sentences. It focuses on matching these consecutive token sequences between candidate and reference sentences, while incorporating a brevity penalty to prevent overly short translations from receiving artificially high scores. Unlike the

BLEU score, which focuses on token-level comparisons, the BERT score uses contextual embeddings to capture semantic similarities between words or full sentences. It has been shown to correlate with human judgment on sentence-level and system-level evaluation. Moreover, the BERT score computes precision, recall, and F1 measure, which can be useful for evaluating different language generation tasks.

In our approach, we use the BERT score as a stochastic checker for hallucination detection. The idea is that if you generate multiple answers from an LLM and there are large variations (inconsistencies) between them, then there is a good chance that these answers are hallucinated. We first generate N random samples (sentences) from the LLM. We then compute BERT scores by comparing each sentence in the original generated paragraph against its corresponding sentence across the N newly generated stochastic samples. This is done by embedding all sentences using an LLM based embedding model and calculating cosine similarity. Our hypothesis is that factual sentences will remain consistent across multiple generations, resulting in high BERT scores (indicating similarity). Conversely, hallucinated content will likely vary across different generations, resulting in low BERT scores between the original sentence and its stochastic variants. By establishing a threshold for these similarity scores, we can flag sentences with consistently low BERT scores as potential hallucinations, because they demonstrate semantic inconsistency across multiple generations from the same model.

Approach 4: Token similarity detection

With the token similarity detector, we extract unique sets of tokens from the answer and the context. Here, we can use one of the LLM tokenizers or simply split the text into individual words. Then, we calculate similarity between each sentence in the answer and the context. There are multiple metrics that can be used for token similarity, including a BLEU score over different n-grams, a ROUGE score (an NLP metric similar to BLEU but calculates recall vs. precision) over different n-grams, or simply the proportion of the shared tokens between the two texts. Out-of-context (hallucinated) sentences should have low similarity with the context.

```
def intersection_detector(
    context: str,
    answer: str,
    length_cutoff: int = 3,
) -> dict[str, float]:
    """
    Check hallucinations using token intersection metrics

    Parameters
    ----------
    context : str
        Context provided for RAG
    answer : str
        Answer from an LLM
    length_cutoff : int
        If no. tokens in the answer is smaller than length_cutoff, return scores of 1.0

    Returns
    -------
    dict[str, float]
        Token intersection and BLEU scores
    """

    # populate with relevant stopwords such as articles
    stopword_set = {}

    # remove punctuation and lowercase
    context = re.sub(r"[^\w\s]", "", context).lower()
    answer = re.sub(r"[^\w\s]", "", answer).lower()

    # calculate  metrics
    if len(answer) >= length_cutoff:
        # calculate token intersection
        context_split = {term for term in context if term not in stopword_set}
        answer_split = re.compile(r"\w+").findall(answer)
        answer_split = {term for term in answer_split if term not in stopword_set}
        intersection = sum([term in context_split for term in answer_split]) / len(answer_spli

        # calculate BLEU score
        bleu = evaluate.load("bleu")
        bleu_score = bleu.compute(predictions=[answer], references=[context])["precisions"]
        bleu_score = sum(bleu_score) / len(bleu_score)

        return {
```

```
            "intersection": 1 - intersection,
            "bleu": 1 - bleu_score,
        }

    return {"intersection": 0, "bleu": 0}
```

Comparing approaches: Evaluation results

In this section, we compare the hallucination detection approaches described in the post. We run an experiment on three RAG datasets, including Wikipedia article data and two synthetically generated datasets. Each example in a dataset includes a context, a user's question, and an LLM answer labeled as correct or hallucinated. We run each hallucination detection method on all questions and aggregate the accuracy metrics across the datasets.

The highest accuracy (number of sentences correctly classified as hallucination vs. fact) is demonstrated by the BERT stochastic checker and the LLM prompt-based detector. The LLM prompt-based detector outperforms the BERT checker in precision, and the BERT stochastic checker has a higher recall. The semantic similarity and token similarity detectors show very low accuracy and recall but perform well with regards to precision. This indicates that those detectors might only be useful to identify the most evident hallucinations.

Aside from the token similarity detector, the LLM prompt-based detector is the most cost-effective option in terms of the number LLM calls because it's constant relative to the size of the context and the response (but cost will vary depending on the number of input tokens). The semantic similarity detector cost is proportional to the number of sentences in the context and the response, so as the context grows, this can become increasingly expensive.

The following table summarizes the metrics compared between each method. For use cases where precision is the highest priority, we would recommend the token similarity, LLM prompt-based, and semantic similarity methods, whereas to provide high recall, the BERT stochastic method outperforms other methods.

The following table summarizes the metrics compared between each method.

Technique	Accuracy*	Precision*	Recall*	Cost (Number of LLM Calls)	Explainability
Token Similarity Detector	0.47	**0.96**	0.03	0	Yes
Semantic Similarity Detector	0.48	0.90	0.02	K***	Yes
LLM Prompt-Based Detector	0.75	0.94	0.53	1	Yes

BERT Stochastic Checker	**0.76**	0.72	**0.90**	N+1**	Yes

*Averaged over Wikipedia dataset and generative AI synthetic datasets
**N = Number of random samples
***K = Number of sentences

These results suggest that an LLM-based detector shows a good trade-off between accuracy and cost (additional answer latency). We recommend using a combination of a token similarity detector to filter out the most evident hallucinations and an LLM-based detector to identify more difficult ones.

Conclusion

As RAG systems continue to evolve and play an increasingly important role in AI applications, the ability to detect and prevent hallucinations remains crucial. Through our exploration of four different approaches—LLM prompt-based detection, semantic similarity detection, BERT stochastic checking, and token similarity detection—we've demonstrated various methods to address this challenge. Although each approach has its strengths and trade-offs in terms of accuracy, precision, recall, and cost, the LLM prompt-based detector shows particularly promising results with accuracy rates above 75% and a relatively low additional cost. Organizations can choose the most suitable method based on their specific needs, considering factors such as computational resources, accuracy requirements, and cost constraints. As the field continues to advance, these foundational techniques provide a starting point for building more reliable and trustworthy RAG systems.

About the Authors

Zainab Afolabi is a Senior Data Scientist at the Generative AI Innovation Centre in London, where she leverages her extensive expertise to develop transformative AI solutions across diverse industries. She has over eight years of specialised experience in artificial intelligence and machine learning, as well as a passion for translating complex technical concepts into practical business applications.

Aiham Taleb, PhD, is a Senior Applied Scientist at the Generative AI Innovation Center, working directly with AWS enterprise customers to leverage Gen AI across several high-impact use cases. Aiham has a PhD in unsupervised representation learning, and has industry experience that spans across various machine learning applications, including computer vision, natural language processing, and medical imaging.

Nikita Kozodoi, PhD, is a Senior Applied Scientist at the AWS Generative AI Innovation Center working on the frontier of AI research and business. Nikita builds generative AI solutions to solve real-world business problems for AWS customers across industries and holds PhD in Machine Learning.

Liza (Elizaveta) Zinovyeva is an Applied Scientist at AWS Generative AI Innovation Center and is based in Berlin. She helps customers across different industries to integrate Generative AI into their existing applications and workflows. She is passionate about AI/ML, finance and software security topics. In her spare time, she enjoys spending time with her family, sports, learning new technologies, and table quizzes.

Like (2) Share

Comments

Log in to comment